

DTSC 2302: Project 2

Ethan Vanbeurden, Rahul Malla, Eric Testen

Part A: The Best Classification Model for Credit Card Defaults

1. Summary of Process and Results

Our objective was to develop a predictive model to identify customers who are likely to default on their credit card payments. We used a dataset from a study that examined the case of customers' default payments in the country of Taiwan. The data involved a total of 30,000 records with 25 features. We implemented a robust EDA process, cleansed the data, engineered features to give us more insight, built machine learning pipelines to reduce data leakage, and performed a rigorous comparison between ten classifier models using cross-validation for each model.

Our evaluation prioritized Recall score for the performance of these models based on their ability to correctly identify as many actual defaulters as possible. The result: a Decision Tree with a maximum depth of 3 proved to be the best model with a mean cross-validation Recall of 68.4% (with a standard deviation of 2.4%). It outperformed more complex models and provided a balance between predictive power and business interpretability.

2. Data Description and EDA (See Appendix A1)

This dataset contains 30,000 non-null observations with a unique identifier that represents individual customers. Our target variable is binary, with 1 for 'default' and 0 for 'no default'. We used ProfileReport imported from ydata_profiling to inspect the data. It showed that the data for

our target variable was moderately imbalanced, with approximately 78% non-defaulters and 22% defaulters. The features included:

- Demographic Data (Age, Sex, Education, and Marital Status)
- Repayment Status History ('PAY_*' features over the course of 6 months)
- Billing ('BILL_AMT*' features capture monthly billing amount over 6 months)
- Payment ('PAY_AMT*' features that capture the monthly amount paid over 6 months).

Our EDA revealed that the most recent repayment status ('PAY_0') was a strong likely predictor of future default, while billing amounts showed high multicollinearity. On closer inspection, the profile report caught anomalous values that did not follow descriptions from the source of the dataset. For example, 'EDUCATION' is a categorical variable with a range of 1 to 4, however, we found values outside of the range. The ProfileReport gave us great information on developing a data cleansing strategy and possible features to engineer for greater clarity.

3. Cleansing and Feature Engineering (See Appendix A2)

We performed several transformations to prepare the data for modelling. We corrected undocumented values in 'EDUCATION' by reassigning 0, 5, and 6 as 4 for 'Other' and 'MARRIAGE' 0 values as 3 for 'Other.' 'SEX' was renamed as 'MALE' and binary-encoded. Finally, the Repayment Status History variables were simplified with anything less than or equal to 0 is 'on-time' while any values greater than 0 would denote the number of months delinquent.

To gain better interpretability of payment behavior, we engineered new features to capture financial health.

- ‘CC_UTIL’ is the current credit card utilization ratio. Calculated by dividing the most recent bill amount with the credit card limit balance. $BILL_AMT1 / (LIMIT_BAL + 1)$. The result is then clipped so that any value below 0 is 0.0 and above 1 is 1.0.
- ‘TOTAL_BILL’ and ‘TOTAL_PAY’ were calculated over 6 months.
- Delinquency summaries such as ‘MIN_DELAY’, ‘MAX_DELAY’, ‘MED_DELAY’, and ‘NUM_DELAY’ (total months delinquent).
- Monthly and overall payment-to-bill ratios to track if a customer is paying in full or just the minimum payment.

Finally, scaling for distance-based models (Logistic Regression, SVM, LDA, QDA, and XGBoost) was applied within each model’s pipeline using StandardScaler to ensure features with large magnitudes (such as payment or billing amounts) did not cause the model to become disproportionate.

4. Models Considered (See Appendix A3)

We evaluated a plethora of models using a Stratified 5-Fold Cross-Validation to ensure stability despite the class imbalance. We experimented with linear models with Logistic Regression as our baseline and Linear Discriminant Analysis. Non-linear models included Quadratic Discriminant Analysis and Support Vector Machines (Linear and RBF kernels). We also used tree and ensemble methods like Random Forest, Decision Trees (with depths 3, 5, and 9), and XGBoost.

We used the parameter ‘class_weight = balanced’ in model pipelines where it was available. This addresses the class imbalance of the target variable by penalizing the false negatives in a model.

5. Our Final Model

Upon initial evaluation of our models, Random Forest was the best performing model with a 65.7% Recall in identifying defaulters, while sacrificing a little bit of the F1 score with 52.4%. However, our Decision Tree with Depth 3 followed closely in second place with a 64.7% Recall and 52.8% F1 score (See Appendix A4).

We ran a StratifiedKFold Cross-Validation of all models using five folds. Each fold is an estimate of Recall on held-out data, or data that is never “seen” during the model's training. Our Random Forest came out with a 63.9% average Recall $\pm$ 1.4% standard deviation. However, the underdog Decision Tree with a Depth of 3, out-paced our Random Forest in the race and won as best model with 68.4% average Recall $\pm$ 2.4% standard deviation. Linear SVM and XGBoost were surprisingly really poor performers (36.1% $\pm$ 1.5% and 36.1% $\pm$ 1.4% respectively). But considering that these models are more optimized for Accuracy instead of Recall explains why this is the case. (See Appendix A5).

This Decision Tree (See Appendix A6) shows a hierarchy of different risk factors by determining the probability of a customer defaulting as it minimizes the Gini Impurity at each split. ‘MAX_DELAY’ is the root of our tree as the most critical feature. A delay of more than 1.5 months in any period is a strong indication that a customer will default. This leads to ‘PAY_0’ (Recent Repayment Status) as a red flag if it’s greater than 1.5 months. When it comes to Total Debt (‘TOTAL_BILL’) and Total Payment Ratio (‘OVERALL_PAY_RATIO’), customers with low total bill amounts are actually sometimes at a higher risk of defaulting. This is possibly due to paying only minimum amounts while higher billed customers can maintain a better standing if they have higher income or credit limits. This model uses Credit Borrowing Power

(‘LIMIT_BAL’) as a secondary filter in the left sub-tree. It suggests that lower credit limits, which are associated with lower income or high risk, correlate with higher default rates.

Examining the high Gini rates for the nodes (ranging from 0.184 to 0.495) tells us there are limits with this model. It shows that there are still difficulties in distributing the two classes. This limitation, we believe, is baked into the raw data itself as the features do not cleanly separate defaulters due to significant overlap between both classes over all the different features. This would require further investigation into implementing other datasets to support this model that capture confounding variables that can potentially indicate defaulters, such as employment, income, health issues, etc.

Overall, despite the limitations, the model demonstrates that a history of behavior, such as delinquent payments, and financial constraints can be more predictive of default than demographic factors like age or gender. The simple nature of this tree makes it easier for banks to easily explain why a customer is flagged for risk of default. This model answers three questions that matter to banks and lenders. How much is owed? How late are repayments? Is there an effort in paying it back?

APPENDIX A: Code

Figure A1: Data Inspection/EDA

Part A: Credit Card Default Model

```
# Import Libraries
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, average_precision_score,
recall_score, classification_report, f1_score, ConfusionMatrixDisplay
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC, LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

from ydata_profiling import ProfileReport
```

Inspect the dataset

```
# Load the dataset
ccd =
pd.read_csv('https://webpages.charlotte.edu/mschuck1/classes/DTSC2301/Data/credit_
card_defaults.csv')
ccd.dropna(inplace=True)
ccd.columns = ccd.columns.str.strip()
ccd['def_pay_next_month'] = ccd['default payment next month']
ccd.drop('default payment next month', axis=1, inplace=True)
ccd.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID                    30000 non-null  int64
1   LIMIT_BAL             30000 non-null  int64
2   SEX                   30000 non-null  int64
3   EDUCATION            30000 non-null  int64
4   MARRIAGE              30000 non-null  int64
5   AGE                   30000 non-null  int64
6   PAY_0                 30000 non-null  int64
7   PAY_2                 30000 non-null  int64
8   PAY_3                 30000 non-null  int64
9   PAY_4                 30000 non-null  int64
10  PAY_5                 30000 non-null  int64
11  PAY_6                 30000 non-null  int64
12  BILL_AMT1             30000 non-null  int64
13  BILL_AMT2             30000 non-null  int64
14  BILL_AMT3             30000 non-null  int64
15  BILL_AMT4             30000 non-null  int64
16  BILL_AMT5             30000 non-null  int64
17  BILL_AMT6             30000 non-null  int64
18  PAY_AMT1              30000 non-null  int64
19  PAY_AMT2              30000 non-null  int64
20  PAY_AMT3              30000 non-null  int64
21  PAY_AMT4              30000 non-null  int64
22  PAY_AMT5              30000 non-null  int64
23  PAY_AMT6              30000 non-null  int64
24  def_pay_nxt_month     30000 non-null  int64
dtypes: int64(25)
memory usage: 5.7 MB

```

```

# EDA Profile Report
eda = ProfileReport(ccd, title="Credit Card Defaults EDA Report: Raw Data",
explorative=True)
eda.to_notebook_iframe()

# eda.to_file(r"C:\Users\ejtes\OneDrive\Desktop\Class Notes & Assignments\2026
Spring\DTSC 2301 & 2302\2302\homework\project2\eda_report_raw_data.html")

```



Overview

Overview

Alerts 39

Reproduction

Dataset statistics

Number of variables	25
Number of observations	30000
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	5.7 MiB
Average record size in memory	200.0 B

Variable types

Numeric	22
Categorical	3

Inspection from EDA Profile Report:

- 25 Variables
- 30,000 Records
- 0 Missing cells
- 0 Duplicates
- High correlation between the most recent pay `PAY_0` and

target variable `def_pay_nxt_month`. A recent pay delinquency is pretty telling.

- `BILL_AMT` columns have high correlation with each other which is expected.

Correlation Heat Map from EDA Profile Report:

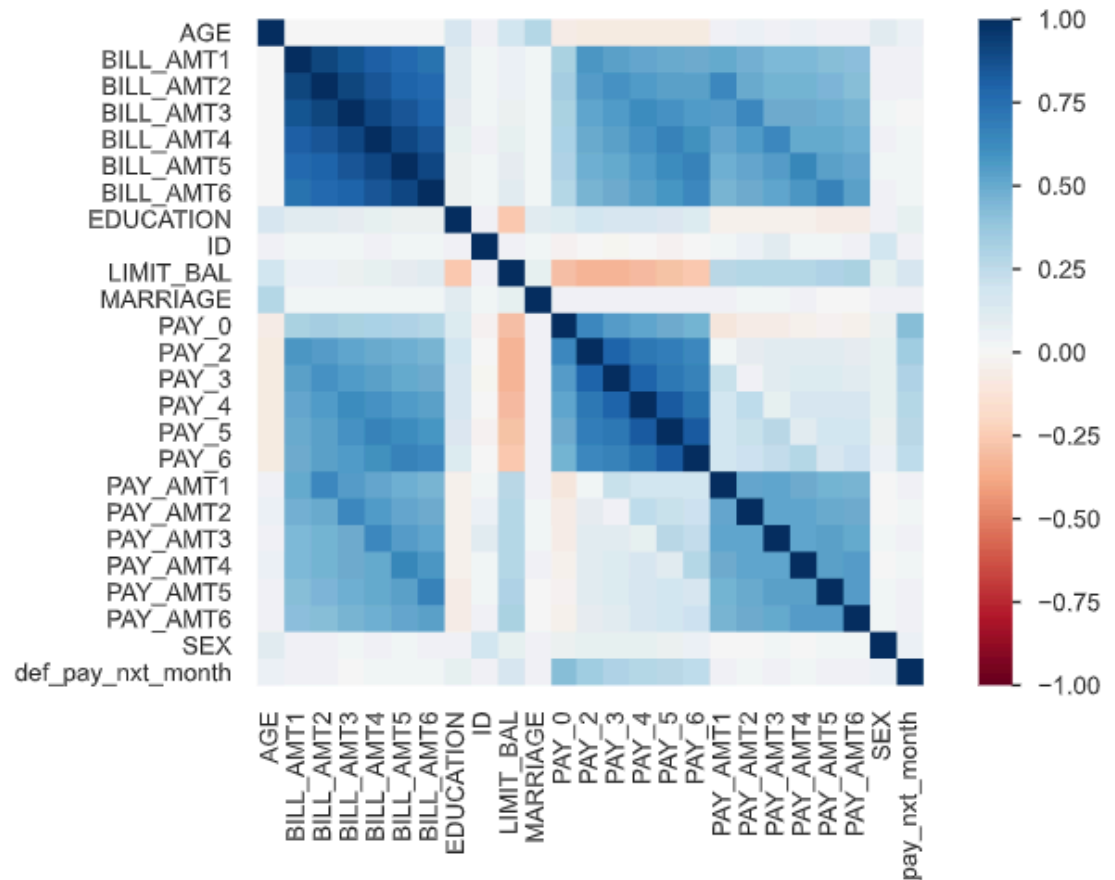


Figure A2: Data Cleansing and Feature Engineering

```
# Drop ID
ccd.drop('ID', axis=1, inplace=True)

# Encode SEX
ccd['MALE'] = ccd['SEX'].apply(lambda x: 1 if x == 1 else 0)
ccd = ccd.drop('SEX', axis=1)

# Encode EDUCATION
ccd['EDU_GRAD_SCHOOL'] = ccd['EDUCATION'].apply(lambda x: 1 if x == 1 else 0)
ccd['EDU_UNIVERSITY'] = ccd['EDUCATION'].apply(lambda x: 1 if x == 2 else 0)
ccd['EDU_HIGH_SCHOOL'] = ccd['EDUCATION'].apply(lambda x: 1 if x == 3 else 0)
ccd['EDU_OTHER'] = ccd['EDUCATION'].apply(lambda x: 1 if x > 3 or x == 0 else 0)
ccd = ccd.drop('EDUCATION', axis=1)

# Encode MARRIAGE
ccd['MARRIED'] = ccd['MARRIAGE'].apply(lambda x: 1 if x == 1 else 0)
ccd['SINGLE'] = ccd['MARRIAGE'].apply(lambda x: 1 if x == 2 else 0)
ccd['MARRIAGE_OTHER'] = ccd['MARRIAGE'].apply(lambda x: 1 if x > 2 or x == 0 else 0)
ccd = ccd.drop('MARRIAGE', axis=1)

# Fix PAY_0 to PAY_6
for col in ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']:
    ccd[col] = ccd[col].apply(lambda x: 0 if x <= 0 else x)

# Create new features
```

```

ccd['CC_UTIL'] = ccd['BILL_AMT1'] / (ccd['LIMIT_BAL'] + 1)
ccd['CC_UTIL'] = ccd['CC_UTIL'].clip(0, 1)

ccd['TOTAL_PAY'] = ccd['PAY_AMT1'] + ccd['PAY_AMT2'] + ccd['PAY_AMT3'] +
ccd['PAY_AMT4'] + ccd['PAY_AMT5'] + ccd['PAY_AMT6']
ccd['TOTAL_BILL'] = ccd['BILL_AMT1'] + ccd['BILL_AMT2'] + ccd['BILL_AMT3'] +
ccd['BILL_AMT4'] + ccd['BILL_AMT5'] + ccd['BILL_AMT6']
ccd['OVERALL_PAY_RATIO'] = ccd['TOTAL_PAY'] / (ccd['TOTAL_BILL'] + 1)

ccd['MIN_DELAY'] = ccd[['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
'PAY_6']].min(axis=1)
ccd['MAX_DELAY'] = ccd[['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
'PAY_6']].max(axis=1)
ccd['MED_DELAY'] = ccd[['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
'PAY_6']].median(axis=1)
ccd['NUM_DELAY'] = ccd[['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5',
'PAY_6']].apply(lambda x: (x > 0).sum(), axis=1)

ccd.head()

```

	LIMIT_BAL	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	PAY_6	BILL_AMT1	BILL_AMT2
0	20000	24	2	2	0	0	0	0	3913	3102
1	120000	26	0	2	0	0	0	2	2682	1725
2	90000	34	0	0	0	0	0	0	29239	14027
3	50000	37	0	0	0	0	0	0	46990	48233
4	50000	57	0	0	0	0	0	0	8617	5670

5 rows × 37 columns

Figure A3: Training and Modelling

```

#Train-test split
X = ccd.drop('def_pay_nxt_month', axis=1)
y = ccd['def_pay_nxt_month']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=32, stratify=y)

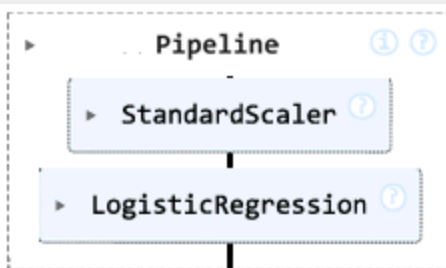
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")

```

Training set size: 24000 samples
Testing set size: 6000 samples

We'll try this on logistic regression first.

```
lr = Pipeline([
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(
        C=1.0,                # inverse of regularization strength. smaller =
                             # more regularization
        class_weight='balanced', # adjusts for the 78/22 target imbalance.
        max_iter=1000,
        random_state=42
    ))
])
lr.fit(X_train, y_train)
```



Decision Tree (Depth 3): no scaling needed for Decision Tree models since these focus on ordering values.

```
dt3 = DecisionTreeClassifier(
    max_depth=3,                # prevents overfitting; unconstrained trees memorize
    min_samples_leaf=30,        # minimum samples in a leaf node
    class_weight='balanced',
    random_state=42
)
dt3.fit(X_train, y_train)
```

Decision Tree (Depth 5)

```
dt5 = DecisionTreeClassifier(  
    max_depth=5,  
    min_samples_leaf=30,  
    class_weight='balanced',  
    random_state=42  
)  
dt5.fit(X_train, y_train)
```

▼ **DecisionTreeClassifier** ⓘ ⓘ

```
DecisionTreeClassifier(class_weight='balanced', max_depth=5,  
    min_samples_leaf=30, random_state=42)
```

Decision Tree (Depth 9)

```
dt9 = DecisionTreeClassifier(  
    max_depth=9,  
    min_samples_leaf=30,  
    class_weight='balanced',  
    random_state=42  
)  
dt9.fit(X_train, y_train)
```

▼ **DecisionTreeClassifier** ⓘ ⓘ

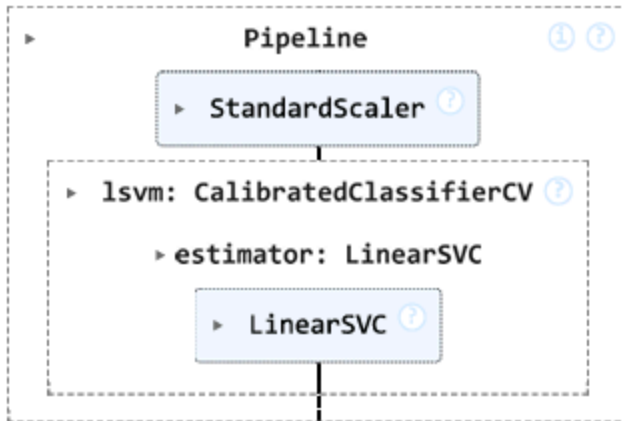
```
DecisionTreeClassifier(class_weight='balanced', max_depth=9,  
    min_samples_leaf=30, random_state=42)
```

Random Forest

```
rf = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=3,  
    class_weight='balanced',  
    min_samples_leaf=30,  
    random_state=42  
)  
rf.fit(X_train, y_train)
```

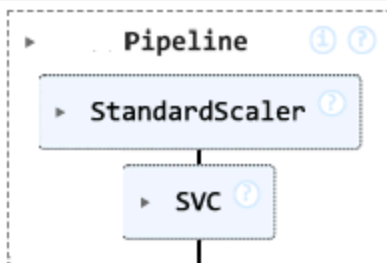
Linear SVM

```
linear_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('lsvm', CalibratedClassifierCV(
        LinearSVC(C=1.0, class_weight='balanced', random_state=42)
    ))
])
linear_svm.fit(X_train, y_train)
```



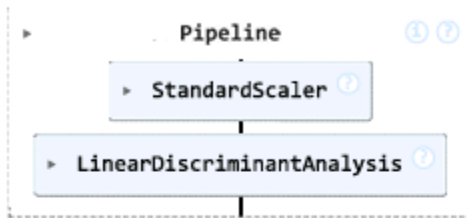
RBF Kernel SVM

```
rbf_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', SVC(
        kernel='rbf',
        class_weight='balanced',
        probability=True,
        random_state=42
    ))
])
rbf_svm.fit(X_train, y_train)
```



Linear Discriminant Analysis (LDA)

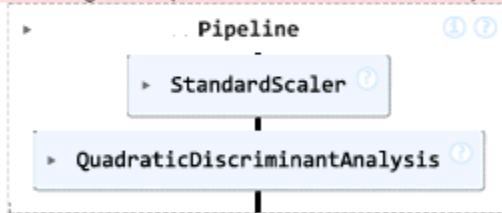
```
lda = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LinearDiscriminantAnalysis(
        solver='svd', # 'svd' does not compute the full covariance matrix;
                      numerically stable
    ))
])
lda.fit(X_train, y_train)
```



Quadratic Discriminant Analysis (QDA)

```
qda = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', QuadraticDiscriminantAnalysis(
        reg_param=0.1 # regularization for the per-class covariance matrices
    ))
])
qda.fit(X_train, y_train)
```

c:\Users\ejtes\anaconda3\Lib\site-packages\sklearn\discriminant_analysis.py:947: Use
Warning: Variables are collinear
warnings.warn("Variables are collinear")



XGBoost

```
xgb = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', XGBClassifier(
        n_estimators=100,
        learning_rate=0.1,
        class_weight='balanced',
        random_state=42,
        eval_metric='logloss'
    ))
])
xgb.fit(X_train, y_train)
```

Figure A4: Initial Model Evaluation

```
def eval(model, X_test, y_test, name, feature_names, top_n=10):
    proba = model.predict_proba(X_test)[: , 1]
    pred = model.predict(X_test)

    print(f"\n{name}")
    print(f" Avg Precision: {average_precision_score(y_test, proba):.3f}")
    print(f" Avg Recall: {recall_score(y_test, pred).mean():.3f}")
    print(f" F1 (default=0.5): {f1_score(y_test, pred):.3f}")
    print(classification_report(y_test, pred, target_names=['No
default', 'Default']))
    confusion_matrix_display = ConfusionMatrixDisplay.from_estimator(model,
X_test, y_test, display_labels=['No default', 'Default'], cmap='Blues')
    plt.title(f"{name} - Confusion Matrix")
    plt.show()

    # Handle both plain models and pipelines
    clf = model[-1] if isinstance(model, Pipeline) else model

    if hasattr(clf, 'estimator'): # CalibratedClassifierCV
        clf = clf.estimator

    if hasattr(clf, 'feature_importances_'):
        importances = clf.feature_importances_
    elif hasattr(clf, 'coef_'):
        importances = abs(clf.coef_[0])
    else:
        print(f" Top features: not available for this model")
        return

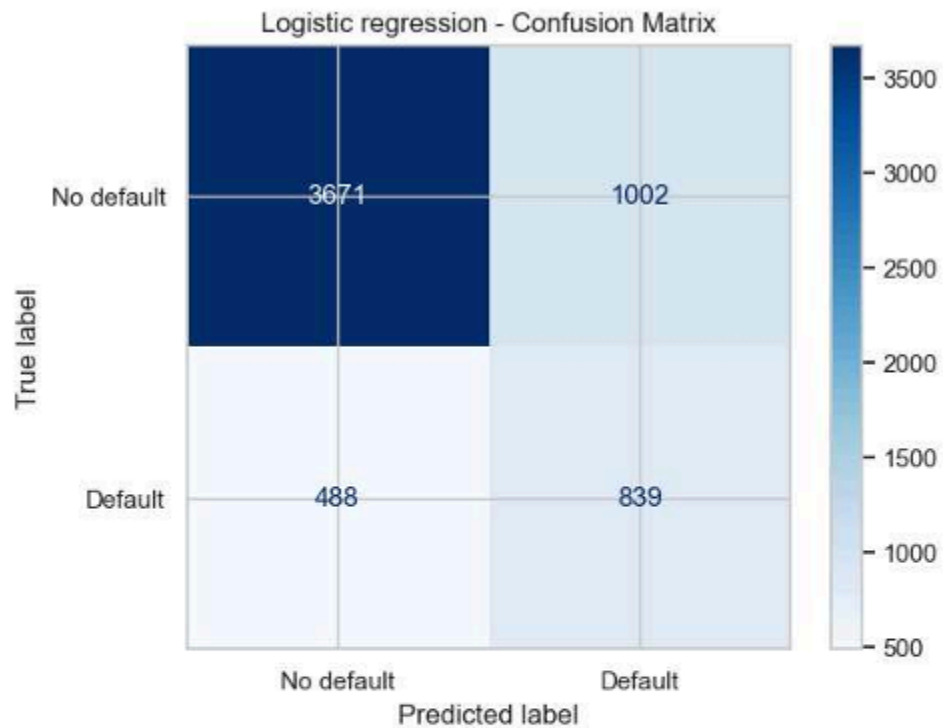
    indices = importances.argsort()[::-1][:top_n]
    print(f"\n Top {top_n} features:")
    for i, idx in enumerate(indices):
        print(f" {i+1}. {feature_names[idx]}: {importances[idx]:.4f}")
```

```
feature_names = list(X_train.columns)

eval(lr, X_test, y_test, 'Logistic regression', feature_names, top_n=10)
eval(dt3, X_test, y_test, 'Decision tree (max_depth=3)', feature_names, top_n=10)
eval(dt5, X_test, y_test, 'Decision tree (max_depth=5)', feature_names, top_n=10)
eval(dt9, X_test, y_test, 'Decision tree (max_depth=9)', feature_names, top_n=10)
eval(rf, X_test, y_test, 'Random Forest', feature_names, top_n=10)
eval(linear_svm, X_test, y_test, 'Linear SVM', feature_names, top_n=10)
eval(rbf_svm, X_test, y_test, 'RBF kernel SVM', feature_names, top_n=10)
eval(lda, X_test, y_test, 'LDA', feature_names, top_n=10)
eval(qda, X_test, y_test, 'QDA', feature_names, top_n=10)
eval(xgb, X_test, y_test, 'XGBoost', feature_names, top_n=10)
```

Logistic regression
 Avg Precision: 0.541
 Avg Recall: 0.632
 F1 (default=0.5): 0.530

	precision	recall	f1-score	support
No default	0.88	0.79	0.83	4673
Default	0.46	0.63	0.53	1327
accuracy			0.75	6000
macro avg	0.67	0.71	0.68	6000
weighted avg	0.79	0.75	0.76	6000



Top 10 features:

1. NUM_DELAY: 0.5710
2. PAY_0: 0.3699
3. MAX_DELAY: 0.3642
4. OVERALL_PAY_RATIO: 0.2897
5. PAY_AMT2: 0.2166
6. LIMIT_BAL: 0.1960
7. BILL_AMT3: 0.1953
8. PAY_2: 0.1903
9. EDU_OTHER: 0.1298
10. PAY_4: 0.1224

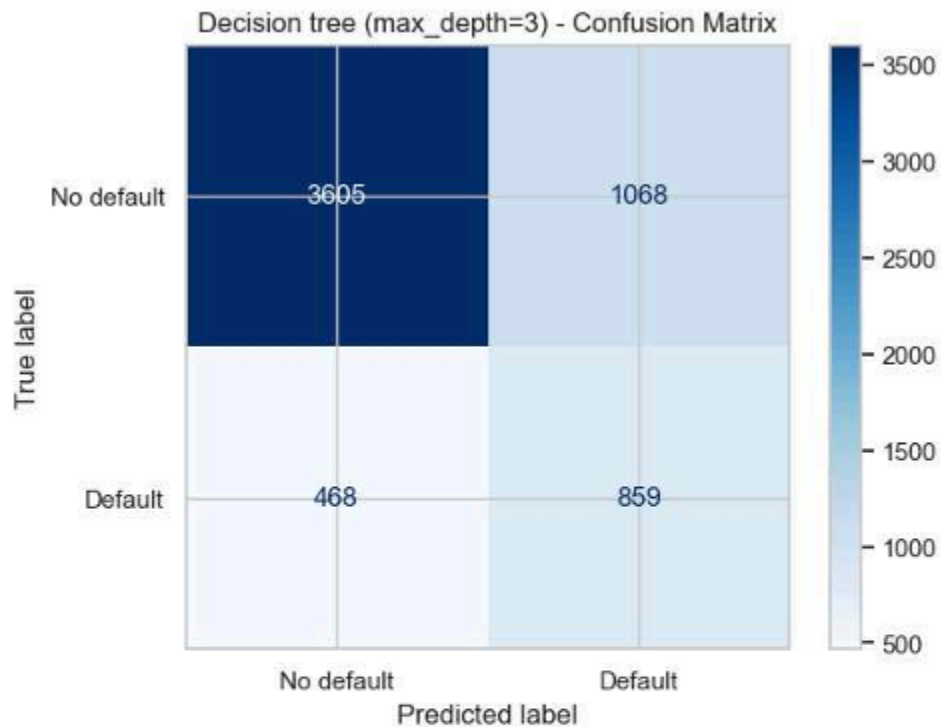
Decision tree (max_depth=3)

Avg Precision: 0.500

Avg Recall: 0.647

F1 (default=0.5): 0.528

	precision	recall	f1-score	support
No default	0.89	0.77	0.82	4673
Default	0.45	0.65	0.53	1327
accuracy			0.74	6000
macro avg	0.67	0.71	0.68	6000
weighted avg	0.79	0.74	0.76	6000



Top 10 features:

1. MAX_DELAY: 0.7077
2. PAY_0: 0.1174
3. TOTAL_BILL: 0.1003
4. NUM_DELAY: 0.0324
5. LIMIT_BAL: 0.0290
6. TOTAL_PAY: 0.0089
7. OVERALL_PAY_RATIO: 0.0043
8. PAY_AMT1: 0.0000
9. AGE: 0.0000
10. MIN_DELAY: 0.0000

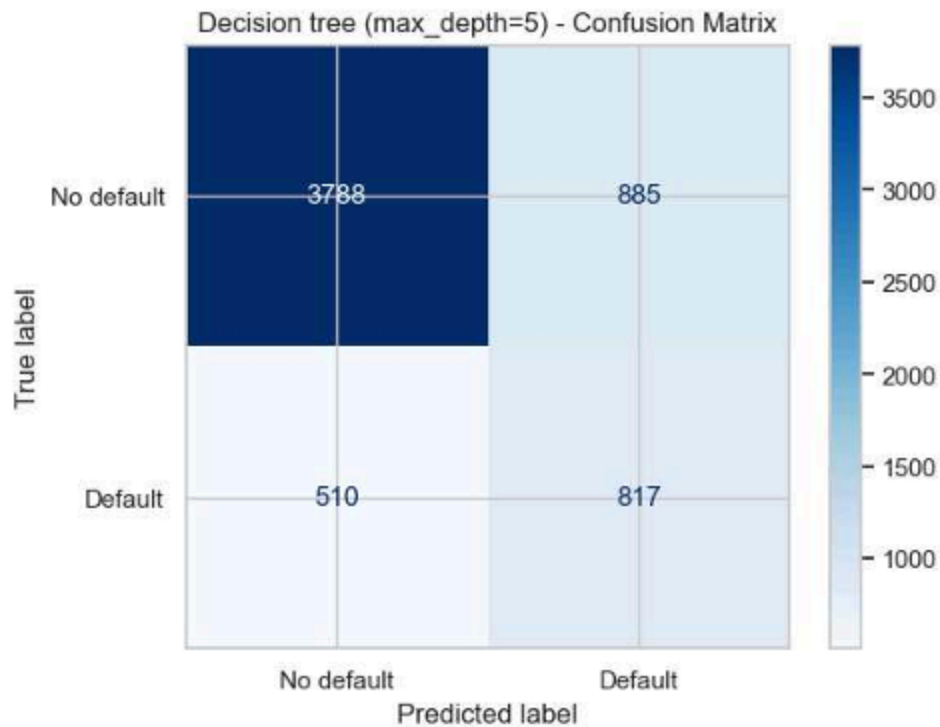
Decision tree (max_depth=5)

Avg Precision: 0.525

Avg Recall: 0.616

F1 (default=0.5): 0.539

	precision	recall	f1-score	support
No default	0.88	0.81	0.84	4673
Default	0.48	0.62	0.54	1327
accuracy			0.77	6000
macro avg	0.68	0.71	0.69	6000
weighted avg	0.79	0.77	0.78	6000



Top 10 features:

1. MAX_DELAY: 0.6394
2. PAY_0: 0.1061
3. TOTAL_BILL: 0.0935
4. LIMIT_BAL: 0.0326
5. NUM_DELAY: 0.0293
6. BILL_AMT1: 0.0228
7. CC_UTIL: 0.0205
8. TOTAL_PAY: 0.0131
9. PAY_AMT1: 0.0113
10. OVERALL_PAY_RATIO: 0.0099

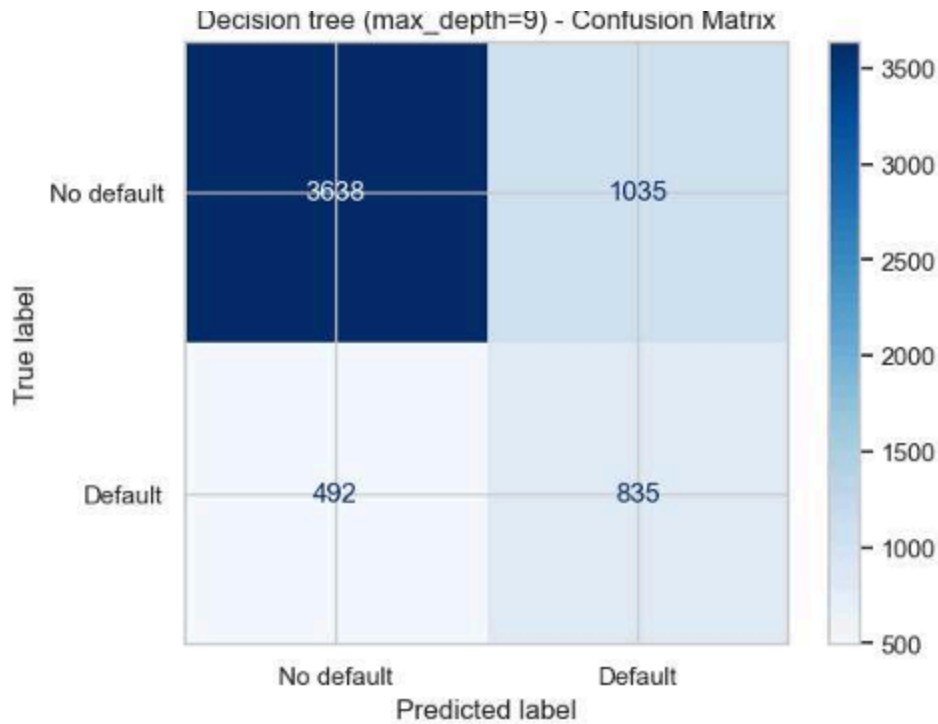
Decision tree (max_depth=9)

Avg Precision: 0.522

Avg Recall: 0.629

F1 (default=0.5): 0.522

	precision	recall	f1-score	support
No default	0.88	0.78	0.83	4673
Default	0.45	0.63	0.52	1327
accuracy			0.75	6000
macro avg	0.66	0.70	0.67	6000
weighted avg	0.78	0.75	0.76	6000



Top 10 features:

1. MAX_DELAY: 0.5303
2. PAY_0: 0.0878
3. TOTAL_BILL: 0.0833
4. CC_UTIL: 0.0370
5. LIMIT_BAL: 0.0361
6. BILL_AMT1: 0.0259
7. NUM_DELAY: 0.0250
8. OVERALL_PAY_RATIO: 0.0239
9. TOTAL_PAY: 0.0185
10. PAY_AMT1: 0.0170

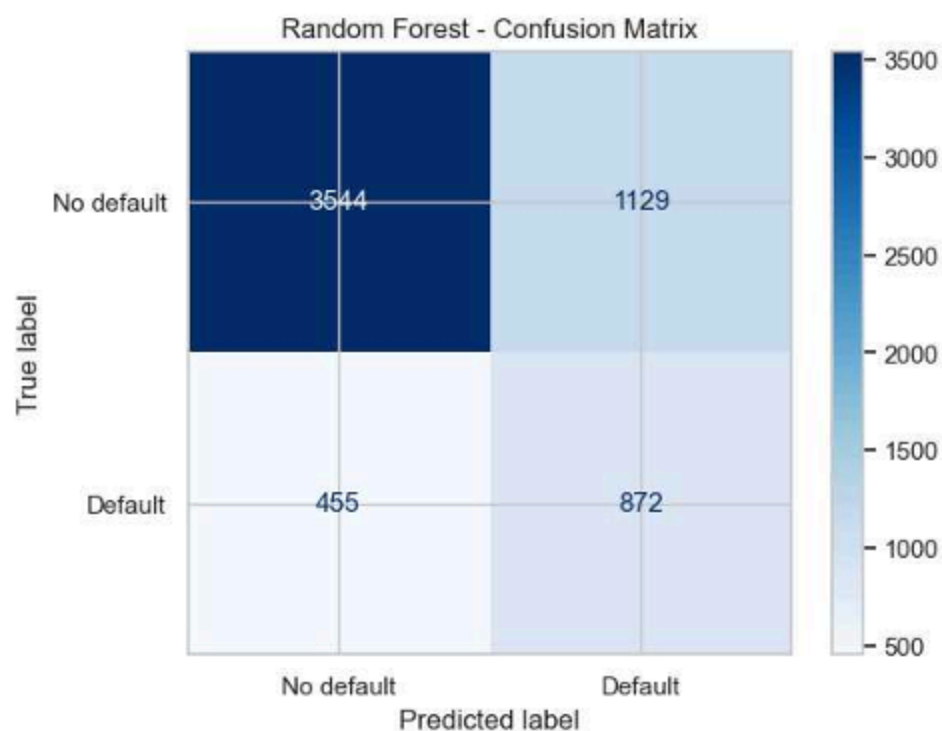
Random Forest

Avg Precision: 0.549

Avg Recall: 0.657

F1 (default=0.5): 0.524

	precision	recall	f1-score	support
No default	0.89	0.76	0.82	4673
Default	0.44	0.66	0.52	1327
accuracy			0.74	6000
macro avg	0.66	0.71	0.67	6000
weighted avg	0.79	0.74	0.75	6000



Top features: not available for this model

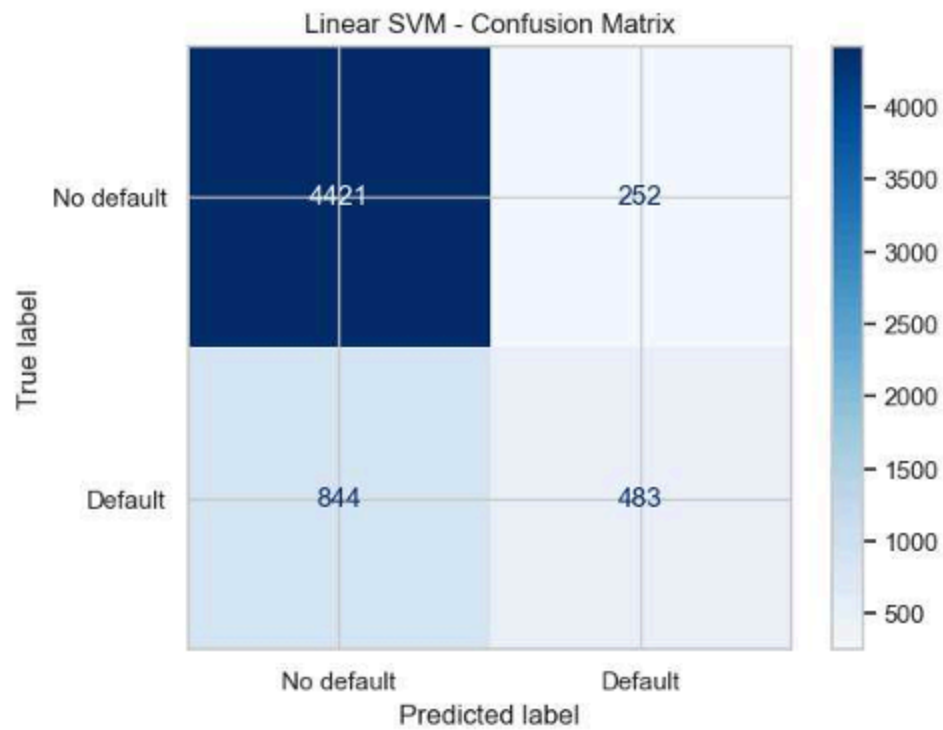
Linear SVM

Avg Precision: 0.539

Avg Recall: 0.364

F1 (default=0.5): 0.468

	precision	recall	f1-score	support
No default	0.84	0.95	0.89	4673
Default	0.66	0.36	0.47	1327
accuracy			0.82	6000
macro avg	0.75	0.66	0.68	6000
weighted avg	0.80	0.82	0.80	6000



Top features: not available for this model

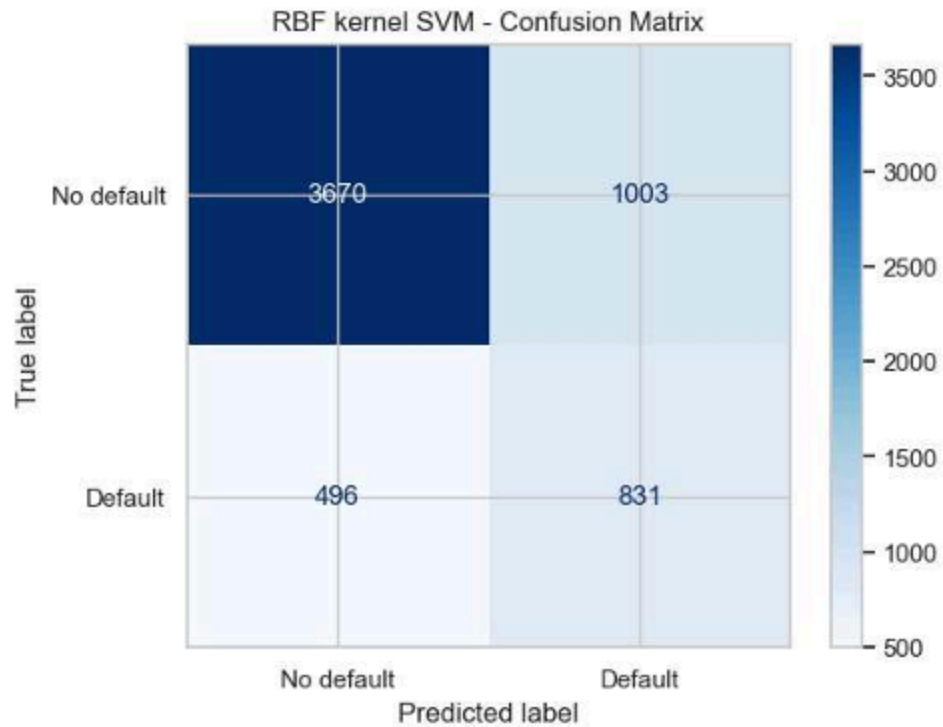
RBF kernel SVM

Avg Precision: 0.472

Avg Recall: 0.626

F1 (default=0.5): 0.526

	precision	recall	f1-score	support
No default	0.88	0.79	0.83	4673
Default	0.45	0.63	0.53	1327
accuracy			0.75	6000
macro avg	0.67	0.71	0.68	6000
weighted avg	0.79	0.75	0.76	6000



Top features: not available for this model

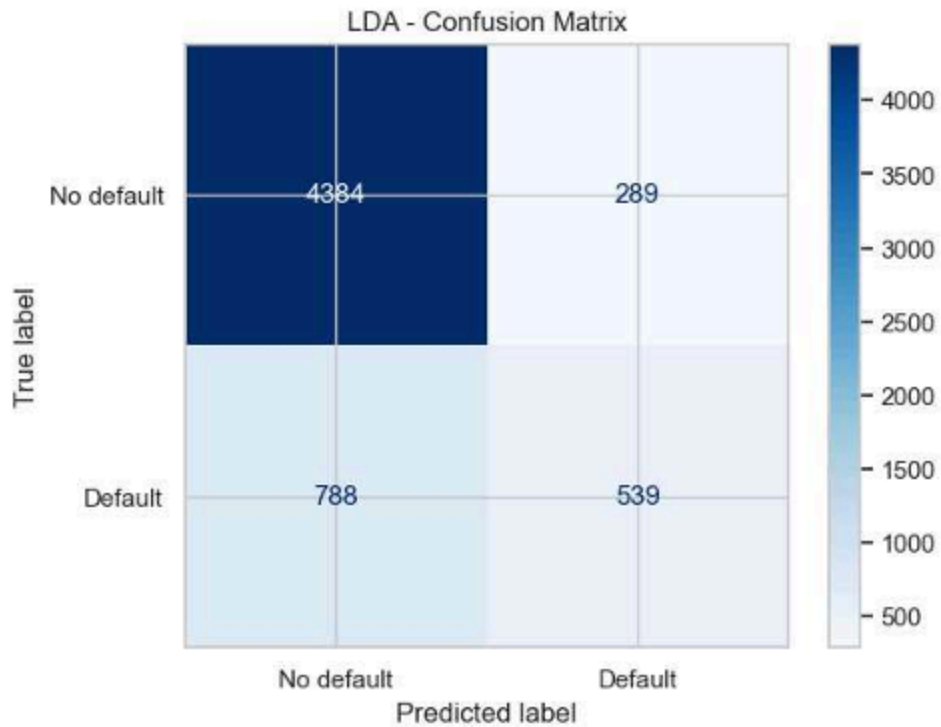
LDA

Avg Precision: 0.538

Avg Recall: 0.406

F1 (default=0.5): 0.500

	precision	recall	f1-score	support
No default	0.85	0.94	0.89	4673
Default	0.65	0.41	0.50	1327
accuracy			0.82	6000
macro avg	0.75	0.67	0.70	6000
weighted avg	0.80	0.82	0.80	6000



Top 10 features:

1. NUM_DELAY: 0.9918
2. PAY_0: 0.5283
3. MAX_DELAY: 0.3388
4. PAY_2: 0.3240
5. MED_DELAY: 0.2160
6. PAY_4: 0.2083
7. PAY_3: 0.2000
8. LIMIT_BAL: 0.1846
9. PAY_5: 0.1337
10. PAY_6: 0.1283

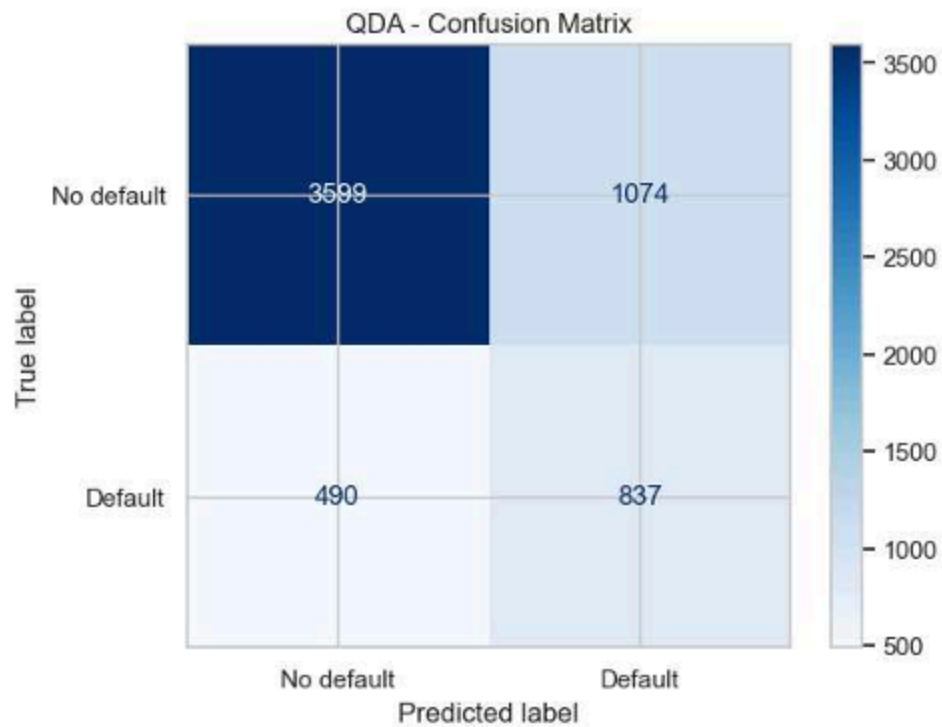
QDA

Avg Precision: 0.519

Avg Recall: 0.631

F1 (default=0.5): 0.517

	precision	recall	f1-score	support
No default	0.88	0.77	0.82	4673
Default	0.44	0.63	0.52	1327
accuracy			0.74	6000
macro avg	0.66	0.70	0.67	6000
weighted avg	0.78	0.74	0.75	6000



Top features: not available for this model

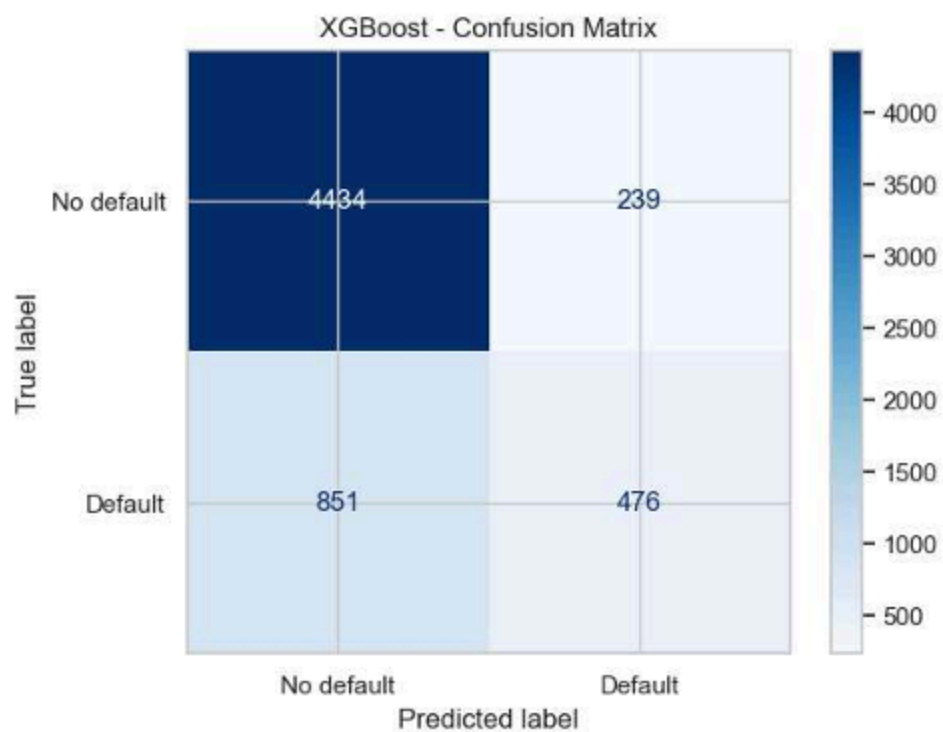
XGBoost

Avg Precision: 0.556

Avg Recall: 0.359

F1 (default=0.5): 0.466

	precision	recall	f1-score	support
No default	0.84	0.95	0.89	4673
Default	0.67	0.36	0.47	1327
accuracy			0.82	6000
macro avg	0.75	0.65	0.68	6000
weighted avg	0.80	0.82	0.80	6000



Top 10 features:

1. MAX_DELAY: 0.3308
2. PAY_0: 0.3078
3. MED_DELAY: 0.0564
4. TOTAL_BILL: 0.0181
5. NUM_DELAY: 0.0174
6. TOTAL_PAY: 0.0137
7. CC_UTIL: 0.0121
8. PAY_2: 0.0109
9. OVERALL_PAY_RATIO: 0.0107
10. BILL_AMT1: 0.0104

Figure A5: Cross-Validation of All Models

Cross-Validation of All Models

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for model, name in [(lr, 'LR'), (dt3, 'DT3'), (dt5, 'DT5'), (dt9, 'DT9'), (rf,
'Random Forest'), (linear_svm, 'Linear SVM'), (rbf_svm, 'RBF SVM'),
                    (lda, 'LDA'), (qda, 'QDA'), (xgb, 'XGBoost')]:
    scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='recall',
n_jobs=-1)
    print(f"{name}: {scores.mean():.3f} ± {scores.std():.3f}")
```

```
LR: 0.612 ± 0.015
DT3: 0.684 ± 0.024
DT5: 0.629 ± 0.011
DT9: 0.629 ± 0.025
Random Forest: 0.639 ± 0.014
Linear SVM: 0.361 ± 0.015
RBF SVM: 0.607 ± 0.018
LDA: 0.399 ± 0.018
QDA: 0.620 ± 0.017
XGBoost: 0.361 ± 0.014
```

Figure A6: Decision Tree with Depth of 3

```
plt.figure(figsize=(20, 8))
plot_tree(
    dt3,
    feature_names=list(X_train.columns),
    class_names=['No Default', 'Default'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree (max_depth=3)")
plt.tight_layout()
plt.show()
```

Decision Tree (max_depth=3)

